Solution: Cyclic Sort

Problem Statement

We are given an array containing `n` objects. Each object, when created, was assigned a unique number from the range `1` to `n` based on their creation sequence. This means that the object with sequence number `3` was created just before the object with sequence number `4`.

Write a function to sort the objects in-place on their creation sequence number in O(n) and without using any extra space. For simplicity, let's assume we are passed an integer array containing only the sequence numbers, though each number is actually an object.

Example 1:

```
Input: [3, 1, 5, 4, 2]
Output: [1, 2, 3, 4, 5]
```

Example 2:

```
Input: [2, 6, 4, 3, 1, 5]
Output: [1, 2, 3, 4, 5, 6]
```

Example 3:

```
Input: [1, 5, 6, 4, 3, 2]
Output: [1, 2, 3, 4, 5, 6]
```

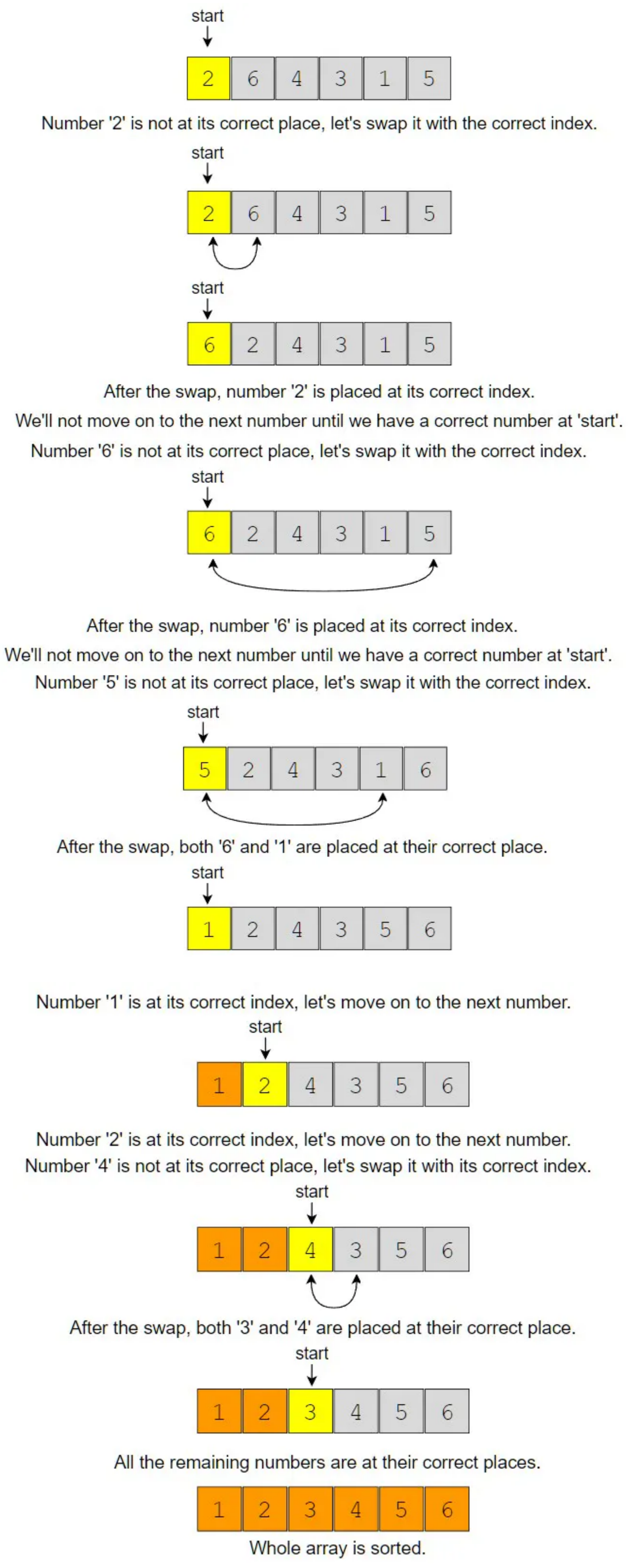
Solution

As we know, the input array contains numbers from the range `1` to `n`. We can use this fact to devise an efficient way to sort the numbers. Since all numbers are unique, we can try placing each number at its correct place, i.e., placing `1` at index `‘0’`, placing `2` at `index ‘1’`, and so on.

To place a number (or an object in general) at its correct index, we first need to find that number. If we first find a number and then place it at its correct place, it will take us $O(N^2)$, which is not acceptable.

Instead, what if we iterate the array one number at a time, and if the current number we are iterating is not at the correct index, we swap it with the number at its correct index. This way, we will go through all numbers and place them at their correct indices, hence, sorting the whole array.

Let’s see this visually with the above-mentioned Example-2:



Number '2' is not at its correct place, let's swap it with the correct index.

After the swap, number '2' is placed at its correct index.
We'll not move on to the next number until we have a correct number at 'start'.
Number '6' is not at its correct place, let's swap it with the correct index.

After the swap, number '6' is placed at its correct index.
We'll not move on to the next number until we have a correct number at 'start'.
Number '5' is not at its correct place, let's swap it with the correct index.

After the swap, both '6' and '1' are placed at their correct place.

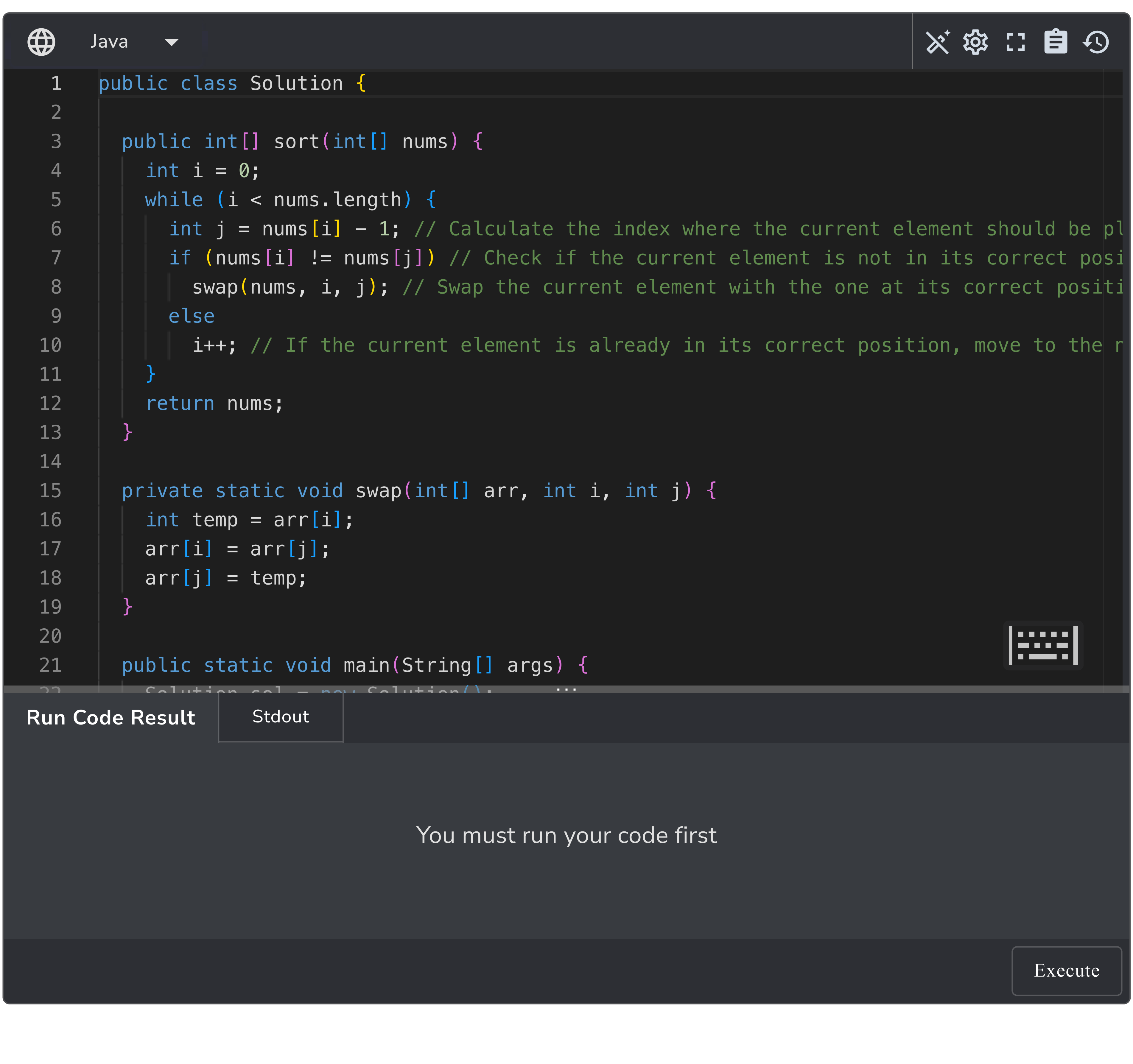
Number '1' is at its correct index, let's move on to the next number.

Number '2' is at its correct index, let's move on to the next number.
Number '4' is not at its correct place, let's swap it with its correct index.

After the swap, both '3' and '4' are placed at their correct place.

All the remaining numbers are at their correct places.

Whole array is sorted.



```java
public class Solution {

  public int[] sort(int[] nums) {
    int i = 0;
    while (i < nums.length) {
      int j = nums[i] - 1; // Calculate the index where the current element should be pl
      if (nums[i] != nums[j]) // Check if the current element is not in its correct posi
        swap(nums, i, j); // Swap the current element with the one at its correct positi
      else
        i++; // If the current element is already in its correct position, move to the r
    }
    return nums;
  }

  private static void swap(int[] arr, int i, int j) {
    int temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
  }

  public static void main(String[] args) {
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is (n). Although we are not incrementing the index `i` when swapping the numbers, this will result in more than `n` iterations of the loop, but in the worst-case scenario, the `while` loop will swap a total of `n-1` numbers, and once a number is at its correct index, we will move on to the next number by incrementing `i`. So overall, our algorithm will take $O(n) + O(n-1)$, which is asymptotically equivalent to $O(n)$.

Space Complexity

The algorithm runs in constant space O(1).

← Previous
Cyclic Sort (Easy)

Next →
Find The Missing Number (Easy)

☐ Mark as Completed